IMPLEMENTING POST-QUANTUM SECURE EXOTIC SIGNATURE SCHEMES IN BLOCKCHAINS

A dissertation submitted to The University of Manchester
for the degree of Master of Science
in the Faculty of Science and Engineering

2026

Royce Steven

Department of Computer Science

Contents

List of Tables

List of Figures

Abstract

Implementing Post-Quantum Secure
Exotic Signature Schemes in Blockchains
Royce Steven
A dissertation submitted to The University of Manchester
for the degree of Master of Science, 2026

Blockchains authenticate transactions with elliptic-curve signatures, which a large quantum computer would break. Basic post-quantum signatures are now standardised, but the *exotic* signatures that give blockchains their advanced features — and in particular *adaptor signatures*, the cryptographic core of atomic swaps and payment channels — remain largely paper-only in the post-quantum setting. This dissertation implements and evaluates one such exotic scheme, LAS, a lattice-based adaptor signature, by reusing the CRYSTALS-Dilithium reference primitives, and demonstrates it end-to-end in a post-quantum atomic swap.

The artefact is built *additively* on Dilithium. The entire lattice core — polynomial arithmetic, the number-theoretic transform, and sampling based on SHAKE (an extendable-output hash function) — is reused unchanged, verified by a file-level diff against a pristine baseline; not a single upstream function was modified. The adaptor layer (PreSign, PreVerify, Adapt, Extract), a bit-packed wire encoding with a validating decoder, and the atomic-swap demonstration are added as new code. The implementation passes functional tests over 1000 iterations on three parameter sets, exact serialization round-trips, a tamper test in which all 4672 single-byte signature flips are rejected, and pinned known-answer tests. An independent second implementation in Rust, built the same way on the Rust ML-DSA crate, reproduces the wire format and the pinned known-answer digest

byte-for-byte and independently confirms the benchmark conclusions.

The primary, fair comparison is LAS against its own simplified Dilithium-style base, using the same parameters and primitives, which isolates the cost of the adaptor layer; the optimised CRYSTALS-Dilithium reference is reported only as context, and a classical Elliptic Curve Digital Signature Algorithm (ECDSA) adaptor as a functionality-matched "price of post-quantum" baseline. Every timing is a mean over repeated independent runs and is reported with its standard deviation. The data show that the adaptor layer is almost free in computation — PreSign, PreVerify, and Adapt each add only single-digit overhead (at most +10.0%) over the basic operation they mirror, at every parameter set, and Extract is the cheapest operation. The real cost is *communication*: the signature is 98.6–99.3% lattice response vector z, and an adaptor swap additionally transmits a public-key-sized statement. The atomic-swap demonstration settles both legs of a two-party, two-chain exchange and asserts that a pre-signature is unspendable until the witness is revealed.

The dissertation concludes that a working, tested, end-to-end post-quantum exotic signature is achievable by reuse of standardised primitives, and identifies tighter packing and migration to the paper's larger modulus as the principal avenues for future work.

Declaration

No portion of the work referred to in this dissertation has been submitted in support of an application for another degree or qualification of this or any other university or other institute of learning.

Copyright

Chapter 1

Introduction

This chapter sets out why post-quantum exotic signatures matter for blockchains, what this project builds, and how the dissertation is organised. It first establishes the context, then reviews the necessary literature, then states the aim and objectives.

1.1 Context and motivation

Public blockchains authorise every transaction with a digital signature, almost always the Elliptic Curve Digital Signature Algorithm (ECDSA), Schnorr, or Boneh–Lynn–Shacham (BLS) signatures over an elliptic curve. The security of all three rests on the hardness of the elliptic-curve discrete logarithm problem. Shor's algorithm solves that problem in polynomial time on a sufficiently large quantum computer [11], so a future quantum adversary could forge signatures and spend other users' funds. "Post-quantum" (PQ) cryptography replaces these assumptions with problems believed hard even for quantum computers, such as those over lattices.

The U.S. National Institute of Standards and Technology (NIST) has standardised *basic* PQ signatures, notably ML-DSA (the Module-Lattice-Based Digital Signature Algorithm), derived from CRYSTALS-Dilithium [9, 4]. A basic signature authenticates a message and nothing more. Blockchains, however, depend on *exotic* signatures that add functionality on top of a basic scheme: multi-signatures, aggregate signatures, threshold signatures, and *adaptor signatures*. These exotic schemes are what give blockchains scalable consensus, compact multi-party authorisation, and advanced payment protocols. A recent survey documents that, while

basic PQ signatures are maturing, their exotic counterparts largely lack efficient, practical implementations in a blockchain setting [3]. Closing that gap — turning paper designs into tested, measured, blockchain-facing artefacts — is the problem this project addresses.

This dissertation focuses on the *adaptor signature* as its exotic scheme. An adaptor signature ties the act of publishing a valid signature to the simultaneous revelation of a secret witness [10, 1]. This single primitive underpins "scriptless" atomic swaps and payment-channel networks: it lets two parties exchange assets, even across different chains, so that either both transfers happen or neither does, without trusted intermediaries or heavy on-chain scripts. In the classical setting these constructions are mature and deployed; in the post-quantum setting they are mostly paper-only, which makes the adaptor signature a representative and high-value exotic scheme to implement and evaluate.

1.2 Subject area and related work

This section reviews only the work needed to understand the problem and justify the approach; it is deliberately not an exhaustive survey, in line with the report guidance to favour depth over breadth.

Exotic signatures for post-quantum blockchains. The motivating survey [3] catalogues exotic signature families — multi-, aggregate, threshold, ring, blind, and adaptor signatures — and identifies the shortage of efficient PQ-secure blockchain implementations as an open challenge. This project contributes one such implementation and evaluation.

Lattice signatures and Dilithium. CRYSTALS-Dilithium is a lattice signature in the "Fiat–Shamir with aborts" family [4, 7]. Its security reduces to the Module Learning With Errors (Module-LWE) and Module Short Integer Solution (Module-SIS) problems. The scheme samples a masked commitment, derives a challenge by hashing, computes a response, and *rejects and retries* when the response would leak the secret — the rejection-sampling step that characterises the family. Its reference implementation provides the polynomial arithmetic, number-theoretic transform (NTT), and SHAKE-based sampling (SHAKE is an extendable-output hash function) that this project reuses.

Adaptor signatures. Adaptor signatures were introduced informally as "scriptless scripts" [10] and later formalised, with applications to channels and swaps [1]. The abstraction adds four algorithms to a base signature — PreSign, PreVerify, Adapt, Extract — relative to a hard relation (Y, y): a pre-signature can be *adapted* into a full signature by anyone who knows the witness y, and publishing the full signature lets anyone holding the pre-signature *extract* y. Anonymous multi-hop locks generalise this to payment paths [8].

Post-quantum adaptor signatures. Esgin, Ersoy and Erkin proposed LAS, a lattice-based adaptor signature built on a Dilithium-style base, together with PQ payment-channel constructions [5]. LAS is the design this project implements. Their work is primarily theoretical (definitions, a construction, and security proofs); a practical, benchmarked, blockchain-facing implementation is what this dissertation contributes. Recent work such as poqeth studies how PQ primitives can be integrated and costed on Ethereum [6], providing a template for the on-chain side.

Classical baseline. The natural answer to "what does post-quantum cost" is a comparison with a *classical adaptor* signature — not merely plain ECDSA. Mature ECDSA-adaptor implementations exist [2]; this project benchmarks against one, and against Dilithium itself, taking care (see chapter 2) to state security parameters explicitly so that the comparisons are fair.

1.3 Project category, aim, and objectives

Category. This is a *systems implementation and evaluation* project grounded in existing research, not a proposal of a new cryptographic protocol. Its emphasis, following the project brief, is practical implementation and benchmarking rather than theoretical design. The contribution is a reliable, tested, and benchmarked realisation of an existing exotic scheme (LAS, [5]) and its demonstration in a blockchain atomic-swap application — comparable in spirit to releasing a working signature library rather than inventing a new scheme. A genuinely new research question would require adding new functionality on top of adaptor signatures (for example, functional adaptor signatures); that is noted as future work but is out of scope here.

Aim. To implement and evaluate a post-quantum exotic signature scheme — a lattice-based adaptor signature reusing the CRYSTALS-Dilithium primitives — and to demonstrate it in a post-quantum blockchain atomic-swap application.

Objectives.

1. Review post-quantum and exotic-signature literature sufficiently to justify selecting the adaptor signature LAS and the Dilithium primitives.
2. Implement LAS additively on the Dilithium reference, reusing the lattice core unchanged and adding the adaptor operations and a byte-level wire encoding.
3. Validate correctness and robustness through functional, serialization, tamper, and known-answer tests, including cross-validation by a second, independent implementation checked against the same known-answer digest.
4. Benchmark LAS fairly — at explicitly stated parameters, with repeated runs and reported variance — against a simplified Dilithium baseline at identical parameters, the NIST-optimised Dilithium, and a classical ECDSA-adaptor baseline, separating computation from communication cost and reporting key-generation time, public-key size, signature size, signing time, and verification time.
5. Demonstrate an end-to-end post-quantum atomic swap using the implementation, and discuss its feasibility.

1.4 Dissertation structure

Chapter 2 describes the methods: the LAS construction, the reuse-based implementation strategy, the testing approach, and the benchmarking methodology. Chapter 3 reports the test and benchmark results and discusses their validity. Chapter 4 evaluates the work against the objectives and reflects on the approach. Chapter 5 concludes and proposes future work.

Chapter 2

Methodology

This chapter explains how the project work was done: the construction implemented, the reuse-based implementation strategy, the testing regime, the independent Rust port that cross-validates the implementation, and — importantly for a fair evaluation — the benchmarking methodology and the parameter choices that make the comparisons defensible. The atomic-swap demonstration and its simulated-ledger model are methodologically light and are described together with their results in section 3.5.

2.1 The LAS construction

LAS is a lattice adaptor signature whose base is a simplified, Dilithium-style Fiat–Shamir-with-aborts signature [5, 4]. All objects live in the ring $R_q = \mathbb{Z}_q[X]/(X^d + 1)$. The public matrix has the form $A = [\, I_n \mid A' \,] \in R_q^{n \times (n+\ell)}$. A key pair is a short secret r and its image $t = A \cdot r$; the hard relation (Y, y) reuses exactly the same structure, so a statement is "just another public key" and recovering its witness is a Module-SIS/LWE problem. Crucially, *key generation is shared*: LAS reuses the basic signature's KeyGen unchanged, and the statement/witness pair (Y, y) is produced by that very same generator. LAS is therefore exactly the basic signature — KeyGen, Sign, Verify — plus the four adaptor operations below, which is why every comparison in this report pairs a LAS operation with the basic operation it extends.

The base signature samples a masked commitment w, forms a challenge $c = H(pk, w, M)$, computes a response, and rejects when $\|\mathbf{z}\|_\infty > \gamma - \kappa$ (eq. (2.1)). The adaptor extension [5] adds four operations relative to a statement Y:

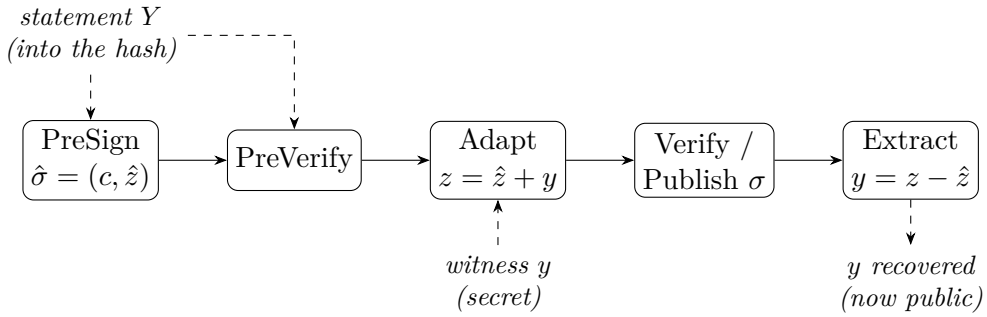

Figure 2.1: The LAS adaptor data flow. The statement Y is folded into the challenge hash during PreSign and PreVerify; the witness y is added by Adapt to turn the pre-signature into a basic signature; and publishing that signature lets anyone holding the pre-signature run Extract to recover y. This last step is what makes an atomic swap atomic.

- PreSign folds the statement into the hash, $c = H(pk, w + Y, M)$, and rejects at the *tighter* bound $\gamma - \kappa - 1$;
- PreVerify recomputes $w' = A\mathbf{z} - c\,t$ and checks $c = H(pk, w' + Y, M)$;
- Adapt adds the witness, $\mathbf{z} \leftarrow \mathbf{z} + y$, producing a signature that the basic verifier accepts;
- Extract recovers $y = \mathbf{z} - \hat{\mathbf{z}}$ from a signature and its pre-signature.

$$\text{accept iff } \|\mathbf{z}\|_\infty \le \gamma - \kappa, \qquad \gamma = \kappa\, d\,(n + \ell). \tag{2.1}$$

The single subtle design point is the bound budget. PreSign must reject at the tighter bound $\gamma - \kappa - 1$. The witness is ternary, so $\|y\|_\infty \le 1$, and after Adapt adds it the response satisfies $\|\mathbf{z}\|_\infty \le \gamma - \kappa$ and clears eq. (2.1). Loosening PreSign to $\gamma - \kappa$ would let adapted signatures exceed the bound, and basic verify would then reject every adapted signature. Figure 2.1 shows the four-operation data flow and, crucially, where the statement Y enters and where the witness y is revealed.

Table 2.1: Source files of the implementation, classified by how the reference is used. The lattice core is reused unchanged; the only modified file is the `Makefile`; the scheme, serialization, application, and evaluation are new.

Category	Files	Role
Reused unchanged	`poly/polyvec/ntt/reduce/rounding`, `packing`, `sign`, `fips202`, `params`	the entire lattice core
Modified	`Makefile`	compile new targets
Added	`las.{c,h}`, `serialize.{c,h}`, `amhl`, `chain`, tests, benchmarks	this work

2.2 Implementation strategy: reuse, do not reinvent

The guiding principle is that an exotic scheme equals a basic scheme plus extra functions, so the lattice arithmetic should be *reused*, not rewritten. The implementation is therefore built additively on the CRYSTALS-Dilithium C reference; the same strategy is later applied a second time, in Rust, on the Rust ML-DSA crate (section 2.5).

Table 2.1 summarises the classification: no upstream source function is modified, so the security-critical lattice arithmetic is exactly the audited reference code, and every line specific to the adaptor scheme is new and isolated. This both reduces the surface for implementation error and makes the boundary between reused and new code unambiguous.

Data types and the matrix product. LAS is a self-contained module (`las.{c,h}`) over the reference's degree-d polynomial type. A key pair is a ternary secret r of $n+\ell$ polynomials and its image $t = A\,r$ of n polynomials; a (pre-)signature is a challenge c and a response $\mathbf{z}$ of $n+\ell$ polynomials; a statement/witness (Y, y) has exactly the key-pair shape. The public matrix has the form $A = [\,I_n \mid A'\,]$, so the product $A\,v = v_{\text{top}} + A' v_{\text{bot}}$ adds the identity block directly and multiplies only A' in the NTT domain. This both saves work and matches exactly how the reference computes As_1 in key generation.

Hash, challenge, and samplers. The random oracle H is built from SHAKE256: it absorbs a canonical encoding of the public key, then the commitment (w for Sign, $w + Y$ for PreSign), then the message, and squeezes a seed. The challenge

sampler is the reference's `SampleInBall` with weight κ (table 2.2; e.g. 60 at the LAS-2020/845 reference set, 49 at the target Simplified Dilithium-III), which guarantees $\|c\|_1 = \kappa$ and $\|c\|_\infty = 1$. Two samplers are added: a ternary sampler for secrets and witnesses (two bits per coefficient), and a uniform sampler over $[-\gamma, \gamma]$ for the mask (an 18-bit rejection sampler). The infinity-norm rejection checks reuse the reference's `poly_chknorm` directly.

Simplifications relative to the full Dilithium scheme. Following the LAS construction [5], the base signature omits the optimisations of the full Dilithium scheme — the hint vector, Power2Round, and high/low-bit decomposition — and hashes the full commitment w rather than its high bits. This keeps the algebra transparent at the cost of larger keys and signatures (quantified in chapter 3). The module depends on no mode-specific constant, so the same code runs at any of the parameter sets used in the evaluation.

Modulus (a deliberate deviation). The original construction specifies $q \approx 2^{24}$. Reusing the reference NTT fixes the root-of-unity table to $q = 8\,380\,417 \approx 2^{23}$, so this implementation uses that modulus. Since $q > 2\gamma$, every intermediate value is represented faithfully and correctness is unaffected; only the concrete Module-SIS/LWE security margin differs from the larger-modulus parameter set. Migrating to $q \approx 2^{24}$ would require a new NTT table and is treated as future work (section 5.2).

2.3 Serialization and the on-chain interface

A deployable scheme exchanges *bytes*, not in-memory structs. A bit-packed wire encoding and a *validating* decoder were implemented, together with a verify-from-bytes entry point (`las_verify_packed`) that an on-chain verifier would consume. The packed field widths (used for the size analysis in chapter 3) are: public key / statement at 23 bits/coefficient, ternary secret / witness at 2 bits/coefficient, and the response z at 18 bits/coefficient.

A verifier cannot trust its input, so the decoder is *defensive*: it rejects a public-key coefficient $\geq q$, the invalid ternary code, and any response field outside the 18-bit band. The encoder is symmetric, rejecting a non-ternary secret or a response exceeding $\gamma - \kappa$. This is the realistic interface a Solidity, precompile, or

zero-knowledge-circuit verifier would expose, and it is the object the tamper test in section 3.1 attacks.

2.4 Testing methodology

Correctness and robustness were established with four test types.

1. **Functional tests** run 1000 iterations per Dilithium mode (2, 3, and 5), each with fresh public parameters, keys, and message. Every iteration runs the full adaptor cycle and asserts: the pre-signature pre-verifies; the pre-signature *fails* basic verify (the statement-binding tripwire); the adapted signature verifies; and the extracted witness equals the original *exactly*. A basic sign/verify round-trip and a one-bit-forgery rejection are also checked. Passing all 1000 iterations at every mode is the bar for functional correctness.

2. **Serialization tests** exercise the byte encoding over 256 random instances, asserting exact round-trip for keys and all signature variants.

3. **A tamper test** flips every one of the 4672 bytes of a valid packed signature, one at a time, and asserts that all 4672 flips break verification; it also checks that the validating decoder rejects malformed bytes.

4. **Known-answer tests** fix all inputs, run the deterministic application programming interface (API), fold the packed bytes of every object into a single SHAKE256 digest, and compare it against a pinned 32-byte value. Any unintended change to key generation, signing, the adaptor algebra, or the serialization flips the digest. The digest is stable across machines and compilers, so an independent on-chain verifier can be cross-checked against it.

All tests pass, and the implementation compiles with no warnings under a strict set of compiler diagnostics.

2.5 An independent second implementation: the Rust port

The complete scheme — the ordinary signature, the four adaptor operations, the wire encoding, and the deterministic known-answer interface — was implemented

a second time, in Rust. The port follows the same reuse principle as the C implementation: it is layered on a fixed version of the `fips204` crate, the Rust implementation of ML-DSA, reusing that crate's polynomial arithmetic, number-theoretic transform, and SHAKE-based sampling, and adding the LAS layer as new, self-contained modules. The wire format is byte-identical to the C encoder's by construction.

The port serves three methodological purposes. First, *cross-validation*: the strongest practical argument that a cryptographic implementation is correct, short of a formal proof, is a second implementation written against the paper rather than against the first implementation's quirks, agreeing on pinned test vectors. Both implementations reproduce the same known-answer digest — a single SHAKE256 hash folded over the packed bytes of every object of four deterministic vectors — so their samplers, algebra, and encoders agree exactly (section 3.3). Second, *an independent benchmark methodology*: the Rust ecosystem's standard statistical harness, Criterion.rs (warm-up, 300 samples per operation, outlier classification, bootstrap confidence intervals), measures the same operations without sharing a line of timing code with the hand-rolled C harness, so agreement between the two defends the C numbers against harness artefacts. Third, *deployment realism*: a memory-safe language is the plausible vehicle for any production use of the scheme, and the port shows the design does not depend on C-specific constructions.

To make the C and Rust timings directly comparable, the two protocol drivers mirror each other exactly: the same repetition scheme, the same fixed public-parameter seed, the same fixed message, the same success-path gating (the correctness contract of section 2.4 is asserted before any timing starts), and the same directly counted rejection-loop statistics. The Criterion harness then provides the independent statistical cross-check on top.

2.6 Benchmarking methodology and fair comparison

The evaluation separates two distinct costs and never conflates them: *computation* cost (wall-clock time per operation) and *communication* cost (bytes transmitted or stored). Each is measured as follows.

Repeatability and machine of record. Every timing is the mean over 5 independent repetitions of 500 iterations per sign-class operation and 1000 iterations per verify-class operation, single-threaded, compiled at `-O3`. Results are reported as mean ± sample standard deviation so that machine and scheduling noise are visible. Two validity gates precede every timing: the full correctness contract of section 2.4 is asserted on the very objects about to be timed (so no failure path is ever measured), and the directly counted rejection-loop attempt rate is asserted against its closed-form prediction within five standard deviations (so a broken sampler cannot produce plausible timings). All measurements — C, Rust, and the classical baseline — were taken on one machine: an AMD Ryzen 7 7745HX (8 cores, 16 threads) with about 7.4 GB of RAM visible to the Linux environment, running Ubuntu 24.04.3 LTS (long-term support; kernel 6.6) under the Windows Subsystem for Linux (WSL2) on Windows, with the GNU Compiler Collection (GCC) 13.3.0 and GNU Make 4.3 for C and stable `rustc` in release profile for Rust. The reference implementation, the `fips204` crate, and the classical baseline library are used at fixed versions, and each reported table lists the exact command that produces it (appendix A.1).

Per-operation reporting, not cumulative. Every operation is timed and reported *independently* — KeyGen, Sign, Verify, PreSign, PreVerify, Adapt, and Extract — rather than as a single cumulative end-to-end time. The reason is that the operations are split across parties and machines: in a swap, one party signs or pre-signs, another verifies and later adapts, and a third may extract, so a combined figure would average over costs that are never actually paid together by one party. Per-operation timing also exposes the adaptor overhead operation by operation, which a cumulative number would hide. This is the primary timing result (table A.1); an end-to-end figure adds nothing the per-operation breakdown does not already give.

Primary (fair) comparison: LAS vs. its own simplified base. LAS is defined over a simplified Dilithium-style base signature (Fiat–Shamir with aborts, ternary secrets, no hint vector); LAS is exactly that base plus the four adaptor operations, built on the same code, the same parameters, and the same primitives. The primary comparison therefore holds everything fixed except the adaptor layer, and measures its *overhead*: this is the only same-algorithm, same-parameter, same-primitive comparison, so any difference it shows is genuinely the cost of

adding adaptor functionality. Each adaptor operation is paired with the base operation it mirrors algorithmically:

- PreSign against the base Sign — both sample a masked commitment, hash, and respond under rejection sampling; PreSign only folds the statement Y into the commitment and uses a tighter rejection bound;
- PreVerify against the base Verify — both recompute the same commitment shape, PreVerify with the extra $+Y$;
- Adapt against the base Verify — Adapt runs a PreVerify and then adds the witness;
- Extract is reported separately, having no base-operation analogue.

Parameter sensitivity. To test whether the adaptor overhead depends on the security level, the same base/adaptor comparison is repeated at several parameter sets. The module dimensions for these sets are taken from the standard Dilithium parameter choices at NIST levels 2, 3, and 5 — used here purely as established, well-studied lattice dimensions to scale the scheme across — together with the dimensions of the original LAS construction (table 2.2). Setting the row count n and extra columns ℓ to a level's (K, L), and the challenge weight κ to its τ, fixes the public-key, secret, and challenge dimensions. The implementation's parameters are compile-time constants, so the identical code runs at every set; the bound $\gamma = \kappa\, d\, (n + \ell)$ and the sampler widths follow from them. The concrete bit-security of each set is not formally claimed, as security proofs are beyond the scope of this work.

Component-level size analysis. Rather than reporting only total object sizes, each key and signature is decomposed into its components (challenge c, response z, statement Y, witness y, pre-signature) so that the *cause* of any size difference can be identified. This breakdown is presented in chapter 3.

Classical baseline (functionality-matched, not security-matched). The second baseline is a classical secp256k1 ECDSA adaptor signature [2]. This is *not* a same-security post-quantum comparison: secp256k1 provides approximately 128-bit *classical* security, is not quantum-resistant, and therefore has no corresponding

Table 2.2: Parameter sets used in the evaluation. The *LAS-2020/845 reference* set is the original paper's parameter set; the *Simplified Dilithium-II/III/V* sets reuse the dimensions of Dilithium modes 2/3/5 (so the public-key, secret, and challenge dimensions line up with NIST levels 2/3/5) and are engineering benchmark settings, *not* formal NIST/ML-DSA security-equivalence claims. All lattice sets share the ring degree $d = 256$ and modulus $q = 8\,380\,417 \approx 2^{23}$ inherited from the reused Dilithium NTT; *Simplified Dilithium-III* is the project's target set. The classical secp256k1 ECDSA adaptor is a functionality-matched baseline at ≈128-bit classical security. Table 2.3 defines each symbol.

Setting	n	ℓ	M	κ	γ	d	q
LAS-2020/845 reference	4	4	8	60	122 880	256	8 380 417
Simplified Dilithium-II	4	4	8	39	79 872	256	8 380 417
Simplified Dilithium-III (target)	6	5	11	49	137 984	256	8 380 417
Simplified Dilithium-V	8	7	15	60	230 400	256	8 380 417
Classical (secp256k1 ECDSA adaptor)	≈128-bit classical (see table 3.3)						

NIST post-quantum level. Instead it is a functionality-matched "price of post-quantum" baseline: both schemes provide adaptor-signature functionality for blockchain-style atomic swaps, but they rely on different hardness assumptions and different algebra (classical discrete-log versus post-quantum lattice). LAS is evaluated as the post-quantum adaptor construction; the ECDSA adaptor represents the compact, widely deployed classical alternative. Plain ECDSA is excluded: the fair counterpart of an adaptor signature is another adaptor signature, not a basic one.

Table 2.3: Security and scheme parameters: the symbol, its meaning, and the value(s) it takes across the parameter sets, *listed in the row order of* table 2.2 (LAS-2020/845 reference; Simplified Dilithium-II; -III; -V). The notation follows the LAS paper [5]: the ring degree is its d, and this build runs at $d = 256$, the ring degree of the reused Dilithium NTT. The figure and table captions in chapter 3 refer back to these names and values.

Symbol	Meaning	Value(s): LAS-2020/845 reference, Simplified Dilithium-II/III/V
n	module rank: rows of A, length of public key t	4, 4, 6, 8
ℓ	extra columns of A' (secret-vector extension)	4, 4, 5, 7
$M = n + \ell$	response dimension (number of polynomials in $z, \hat{z}, y$)	8, 8, 11, 15
κ	challenge Hamming weight (number of ± 1 in c; Dilithium τ)	60, 39, 49, 60
γ	rejection / masking bound, $\gamma = \kappa\, d\, (n + \ell)$	122 880, 79 872, 137 984, 230 400
d	ring degree $(X^d + 1)$	256 (all sets)
q	modulus from the reused NTT, $q \approx 2^{23}$	8 380 417 (all sets)

Chapter 3

Results and discussion

This chapter reports what was built and measured, and discusses why the results should be believed. It covers correctness testing, then computation cost, then communication cost with the component breakdown, and finally the application demonstration.

3.1 Correctness and robustness

Before any performance claim, the implementation must be shown to be *correct*. An adaptor signature is not correct merely because it signs and verifies; it must satisfy a specific contract, and a consolidated harness (`test_contract`) checks every clause of that contract and prints each as a labelled pass. Table 3.1 lists the checks and their outcomes.

Each clause matters. Clauses 1–4 are the adaptor contract itself: a pre-signature is verifiable yet unspendable (the tripwire, clause 2), becomes a basic signature once the witness is added (clause 3), and the act of completing it reveals the witness (clause 4) — the mechanism that makes an atomic swap atomic. Clauses 5–6 are robustness: the byte-level verifier an on-chain integration would consume must not trust its input, so it rejects tampering and malformed encodings. Clause 7 is reproducibility, which lets an independent verifier be cross-checked against fixed vectors. All clauses pass, with zero compiler warnings under the project's strict flags.

Table 3.1: The adaptor-signature correctness contract, each clause checked and passed by `test_contract` (corroborated at scale by the 1000-iteration functional suite on modes 2/3/5, the 256-instance serialization suite, and the pinned known-answer test).

#	Property	Result
1	PreSign produces a pre-signature that PreVerify accepts	pass
2	the pre-signature *fails* basic Verify (statement-binding tripwire)	pass
3	Adapt yields a signature that basic Verify accepts	pass
4	Extract recovers the witness *exactly* ($y' = y$)	pass
5a	a tampered message fails Verify	pass
5b	every one of the 4672 single-byte flips of the packed signature is rejected	pass
6	malformed packed bytes are rejected ($pk \geq Q$, ternary code-3, z out of band)	pass
7	the deterministic API is byte-identical on re-run	pass

3.2 Computation cost

The primary computation result is the cost of the *adaptor layer* — what LAS adds to the basic signature it is built on. This is the base-signature path (SIGN and VERIFY, with $c = H(pk, w, M)$ and no statement) against the LAS adaptor path (PRESIGN, PREVERIFY, ADAPT, EXTRACT, with $c = H(pk, w + Y, M)$); the two paths share the same algorithm, parameters, and primitives, so any difference is purely the price of adding adaptor functionality (section 2.6). Each operation is timed *independently*, because in a swap one party signs or pre-signs while another verifies and later adapts, so a single end-to-end figure would be unrealistic.

Figure 3.1 is the primary result. Each LAS adaptor operation (orange) is drawn beside the basic operation it mirrors (blue), with the overhead printed above the orange bar, so the headline is read directly off the figure rather than inferred from two distant groups: at the target setting, *Simplified Dilithium-III*, every adaptor operation stays within single digits of its basic counterpart, and Extract — which has no basic analogue — is the cheapest operation of all. Plotting on an absolute μs scale also makes the shape visible — every operation completes in well under two milliseconds, and signing and pre-signing dominate because of rejection sampling, while the verify-class operations and Extract are far cheaper. The exact means and standard deviations behind the figure are tabulated in table A.2 (appendix). The same base-versus-adaptor comparison repeated across all four parameter sets is a scaling check, deferred to fig. A.1 (appendix).

The adaptor layer is therefore almost free in computation, and the numbers are small *for structural reasons* (overhead at the target setting against the paired basic operation):

- **PreSign ≈ Sign** (+5.6%): PreSign runs the same masked-commit, hash, respond, reject loop as Sign; it only adds the statement Y into the Fiat–Shamir commitment and rejects at a slightly tighter bound. Neither changes the asymptotic work.
- **PreVerify ≈ Verify** (+0.4%): PreVerify recomputes the same commitment shape as Verify, differing only by the $+Y$ addition before hashing.
- **Adapt ≈ Verify** (+3.8%): Adapt first runs a PreVerify and then adds the witness vector, so it costs about one verification plus a few polynomial additions.
- **Extract is cheapest and has no basic analogue**: it only subtracts $z - \hat{z}$ and checks $A\,y = Y$.

This is the headline of the standalone-signature evaluation: adding adaptor functionality to the basic lattice signature costs only single-digit percentages of extra computation, and the two operations unique to an adaptor signature, Adapt and Extract, cost no more than a verification each. As a robustness check that this is not an artefact of one parameter choice, the same base-versus-adaptor comparison was repeated at four parameter sets (the LAS-paper dimensions and the Simplified Dilithium-II/III/V dimensions); the adaptor premium stays in the single digits at every one (fig. A.1, appendix).

Signing and pre-signing are dearer than verification because they use rejection sampling. The acceptance rate was measured directly, by counting rejection-loop attempts rather than estimating from a timing ratio: at the target setting 35.7% of Sign attempts and 37.6% of PreSign attempts are accepted, so a signature takes about 2.799 attempts on average (2.658 for a pre-signature), and the other settings behave alike. This matches the closed-form prediction: one attempt is accepted only if all $(n + \ell) \cdot d$ response coefficients fall within the bound, giving a per-attempt acceptance of $(1 - \kappa/\gamma)^{(n+\ell)d} \approx e^{-1} \approx 36.8\%$. Every benchmark run asserts the measured attempt count against this prediction (within five standard deviations) before any timing is reported, so a broken sampler cannot silently

Table 3.2: Per-operation timing of the two implementations at the target setting, *Simplified Dilithium-III* (μs per operation). The C and Rust columns come from the two protocol drivers, which deliberately mirror each other (same repetition scheme of 5 repetitions $\times$ 500/1000 iterations, fixed public-parameter seed, fixed message, same success-path gating), so they are directly comparable. The final column is the independent Criterion.rs statistical harness (300 samples per operation, bootstrap 95% confidence interval of the mean); its hot-loop protocol yields lower absolute times, and it is included to show that the *relative* conclusions do not depend on the hand-rolled driver.

Operation	C reference (μs)	Rust port (μs)	Rust / C	Rust, Criterion (μs, 95% CI)
KeyGen	45.2 ± 0.2	46.1 ± 0.4	1.02	36.7 [36.6, 36.7]
Sign	520 ± 21	474 ± 10	0.91	419 [417, 420]
Verify	115 ± 1	95.2 ± 1.4	0.83	76.0 [75.9, 76.1]
PreSign	549 ± 31	494 ± 7	0.90	423 [422, 425]
PreVerify	115 ± 2	96.0 ± 1.1	0.83	76.6 [76.6, 76.7]
Adapt	119 ± 2	98.2 ± 1.2	0.82	78.1 [78.1, 78.2]
Extract	40.9 ± 0.5	34.2 ± 0.4	0.84	25.7 [25.6, 25.7]

produce plausible-looking numbers. Rejection sampling is intrinsic to the Fiat–Shamir-with-aborts family, not a defect of this implementation.

3.3 Cross-implementation check: C reference versus Rust port

The scheme exists twice, in C and in Rust (section 2.5), and the two implementations agree at every level at which they can be compared. At the byte level the agreement is exact: both produce the same packed sizes at the target setting (public key 4416 bytes, signature 6752 bytes, checked programmatically on every Rust run) and the same pinned known-answer digest (`641a176c...5a19`), a single SHAKE256 hash folded over every object of four fully deterministic test vectors. Since the digest covers key generation, signing, the four adaptor operations, and the serialization, a divergence anywhere in either implementation's sampling, algebra, or encoding would flip it; its equality is therefore a strong, cheap argument that the two codebases compute the same scheme.

Timing agreement is necessarily looser, since the two builds go through different compilers (GCC versus rustc/LLVM), but it is close: table 3.2 shows every

operation agreeing within about 18% (the largest gap is Adapt), with the Rust port slightly faster on the verify-class operations and slightly slower on key generation. More importantly, every *conclusion* reproduces. The Rust port measures the adaptor overhead at +4.3% (PreSign vs Sign), +0.8% (PreVerify vs Verify), and +3.2% (Adapt vs Verify) — single digits, like the C values of +5.6%, +0.4%, and +3.8% — and its directly counted rejection rates (2.68 attempts per signature, 2.77 per pre-signature) pass the same five-standard-deviation gate against the closed-form prediction. The Criterion harness, with a wholly independent measurement protocol, gives lower absolute times but the same ordering and the same sign-class-versus-verify-class shape. The headline results of this chapter are therefore properties of the algorithm, not artefacts of one codebase, one compiler, or one timing harness.

3.4 Communication cost and component breakdown

The computation results show the adaptor layer is cheap. The size results show where the real cost lies. Figure 3.2 decomposes every LAS object into bytes at the target setting; the aim is to explain *which* component drives the size, not merely to state that signatures are large. The exact bytes at every parameter set are tabulated in table A.3 (appendix), and an object-by-object reference for the target setting in table A.4 (appendix).

Two points follow, and together they give the sharp conclusion of the evaluation (fig. 3.2):

- **The response z is essentially the entire signature** — 98.6% at Simplified Dilithium-II, rising to 99.3% at Simplified Dilithium-V. The challenge c is a fixed 64 bytes and is irrelevant to size. Any optimisation of LAS signatures is an optimisation of z.
- **An adaptor swap additionally transmits a public-key-sized statement Y and a signature-sized pre-signature**, while the witness is small. These are the only objects an adaptor protocol sends that a basic signature does not.

The sharp conclusion: the cost of LAS is not adaptor computation but communication — specifically the response vector z and the statement

Y. The computation results showed the adaptor operations add at most +10.0% of compute over the basic scheme; this section shows the objects that move on the wire are one to a few kilobytes, dominated by z. For a blockchain setting, where every byte is stored and replicated, this is the property that matters, and it is a far more useful statement than "the signature is 6752 bytes".

The response is stored at its full width: each coefficient occupies 18–19 bits (table A.3) rather than being range-reduced or entropy-coded. This is the clearest opportunity for reducing the on-wire footprint, since compressing z would shrink the signature, pre-signature, and settlement payload together; it is left as future work (section 5.2).

3.4.1 The price of post-quantum: classical adaptor baseline

The second baseline is a classical secp256k1 ECDSA adaptor signature [2]. It is the natural functional counterpart of LAS: both are adaptor signatures with the same operation set (KeyGen, Sign, Verify, PreSign, PreVerify, Adapt, Extract) and both drive scriptless atomic swaps. They are compared on equal footing functionally, while differing in their security foundation: secp256k1 provides approximately 128-bit *classical* security and is not quantum-resistant, whereas LAS rests on lattice assumptions believed to resist quantum attack. The comparison therefore measures the practical cost of moving from a classical adaptor signature to a post-quantum one. Plain ECDSA is not used, because the counterpart of an adaptor signature is another adaptor signature.

For this baseline LAS is instantiated at the *Simplified Dilithium-II* parameters, and only that setting is used. The reason is that secp256k1 offers roughly 128-bit classical security, and the nearest post-quantum target is NIST level 2 (also pitched at the 128-bit category); aligning LAS to the Dilithium-II dimensions therefore gives the most like-for-like security target for the classical scheme. Comparing the 128-bit classical adaptor against LAS at the Simplified Dilithium-III or -V settings would pit it against a deliberately stronger post-quantum configuration and would understate the classical scheme, so the higher settings are reserved for the parameter-sensitivity study of section 3.2 and not used here. Table 3.3 reports both schemes at their adaptor operations.

This baseline compares two adaptor-signature schemes with the same high-level functionality but different security assumptions: classical discrete-log security

Table 3.3: Classical vs. post-quantum adaptor signature: same functionality, different security foundation. Classical secp256k1 (≈128-bit *classical* security) versus LAS at the *Simplified Dilithium-II* parameters ($n = 4$, $\ell = 4$, $M = 8$, $\kappa = 39$, $\gamma = 79\,872$, $d = 256$, $q = 8\,380\,417$). The Dilithium-II dimensions are an engineering match to the ≈128-bit (NIST level 2) target, *not* a formal security-equivalence claim (table 2.2); the stronger LAS settings are reserved for the parameter-sensitivity study (section 3.2) and would overstate the post-quantum cost here. Timings μs/op (classical: mean ± SD over 10 runs × 1000 iterations; LAS: 5 repetitions × 500/1000 iterations; same machine); sizes in bytes. KeyGen also produces the statement/witness, which take the public-key/secret-key size. The classical pre-signature is a syntactically distinct object (an ECDSA signature plus a discrete-logarithm-equality proof), whereas the LAS pre-signature shares the signature format.

	ECDSA adaptor (classical)	LAS adaptor (post-quantum)
Computation (μs/op)		
KeyGen	12.8 ± 0.1	29.9 ± 0.1
Sign	17.5 ± 0.2	351 ± 8
Verify	26.4 ± 1.4	73.0 ± 0.9
PreSign	80.7 ± 1.4	353 ± 12
PreVerify	103 ± 1	74.4 ± 0.5
Adapt	1.4 ± 0.01	76.1 ± 0.2
Extract	14.2 ± 0.1	25.0 ± 0.1
Communication (bytes)		
Public key / statement	33	2944
Secret key / witness	32	512
Signature	64	4672
Pre-signature	162	4672

versus post-quantum lattice security. Two findings stand out. First, the post-quantum price is paid almost entirely in *size*: LAS's signature and public key are roughly 73× and 89× larger than the classical adaptor's, whereas in computation every LAS operation stays well under a millisecond — even the largest per-operation factor (Adapt, about 56×, against a classical operation that is nearly free) costs under a tenth of a millisecond in absolute terms. Second, the two schemes pay their *adaptor* overhead in opposite ways. The classical ECDSA adaptor must attach a discrete-logarithm-equality proof, so its PreSign and PreVerify are several times its own basic sign and verify; LAS folds the statement into a hash it already computes, so its adaptor operations stay within a few percent of its base operations. In computation, then, the lattice adaptor layer is relatively cheaper to add than the classical one; the post-quantum cost surfaces

in communication, consistent with the component analysis above.

3.5 Application demonstration

The implementation was demonstrated in an end-to-end post-quantum atomic swap. Two parties, Alice and Bob, swap coins across two ledgers with no trusted third party and no on-chain scripts. Bob draws a fresh statement/witness pair (Y, y) and sends only Y. Each party pre-signs its outgoing transaction against the *same* Y and checks the other's pre-signature; at this point neither pre-signature is spendable. Bob, who knows y, adapts and publishes his signature to claim Alice's coin; the act of publishing reveals y, which Alice extracts and uses to adapt and publish her own signature, claiming Bob's coin. Either both legs settle or neither does. The demo prints this narrative and asserts every step, including the counterfactual that a pre-adaptation signature fails basic verification.

A small ledger abstraction takes this closer to a real chain: accounts with balances, a block height, and adaptor-locked contracts (the scriptless analogue of a hash-time-locked contract, with the hash lock replaced by the statement Y and the time lock by a timeout height). Three scenarios are asserted: a cross-chain swap (happy path), a timeout/refund path in which a premature refund is rejected and both parties recover their funds after the timeout, and a multi-hop payment.

As a bonus-tier check on deployability, the byte-level verifier was also costed on a real Solidity stack: a native on-chain LAS verification measures at approximately 12 million gas, about 40% of Ethereum's 30 million per-block limit. It therefore fits within a block, though at a cost that motivates the precompile or zero-knowledge-proof routes discussed in section 5.2. This on-chain measurement and the multi-hop variant are bonus-tier evidence; they support, but do not displace, the first-stage signature results above.

3.6 Threats to validity

Several factors qualify the results, and are stated here so the reader can judge how far to trust them. First, all timings are machine-dependent; this is mitigated by reporting the standard deviation, fixing one machine and compiler, emphasising ratios over absolute values, and by the cross-implementation check (section 3.3), which reproduces every relative conclusion under a different compiler and an

independent timing harness. Second, the LAS parameter set's concrete security level is not formally established, because security proofs are out of the project's scope; every cross-scheme size and speed claim is therefore qualified by the explicit parameters in table 2.2, and the fair comparison deliberately holds parameters fixed between LAS and its simplified-Dilithium base. Third, the modulus is $q \approx 2^{23}$, inherited from the reused NTT table, rather than the paper's 2^{24}; correctness is unaffected ($q > 2\gamma$) but the security margin differs. Fourth, the extracted witness is exact in this parameterisation, whereas the paper's relaxed setting allows witness noise that can grow across long payment-channel paths; the exact-extraction choice sidesteps that growth for the demonstration but is a simplification worth noting.

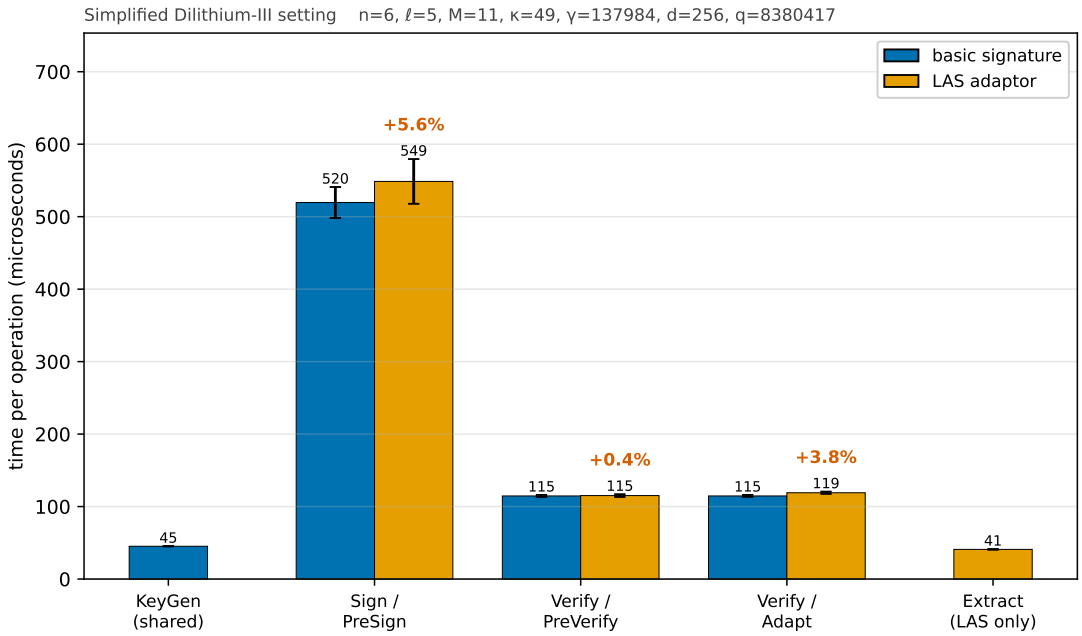

Figure 3.1: The primary computation result, shown visually: each LAS adaptor operation (orange) paired beside the basic operation it mirrors (blue), at the target setting *Simplified Dilithium-III* (its parameters are annotated above the plot). The percentage above each orange bar is the adaptor overhead over its blue partner; exact means and standard deviations are in table A.2 (appendix). KeyGen, Sign, and Verify are reused unchanged by LAS, so KeyGen is shown once (shared) and Sign/Verify are the blue partners of PreSign/PreVerify/Adapt; Extract has no basic analogue and is shown LAS-only. The absolute μs scale also shows the shape: every operation is sub-two-milliseconds and the rejection-sampling operations (Sign, PreSign) dominate, while the verify-class operations and Extract are far cheaper. Error bars are the sample standard deviation over 5 repetitions of 500 (sign-class) / 1000 (verify-class) iterations on the machine of record (section 2.6); the same comparison across all four parameter sets is the scaling check in fig. A.1 (appendix).

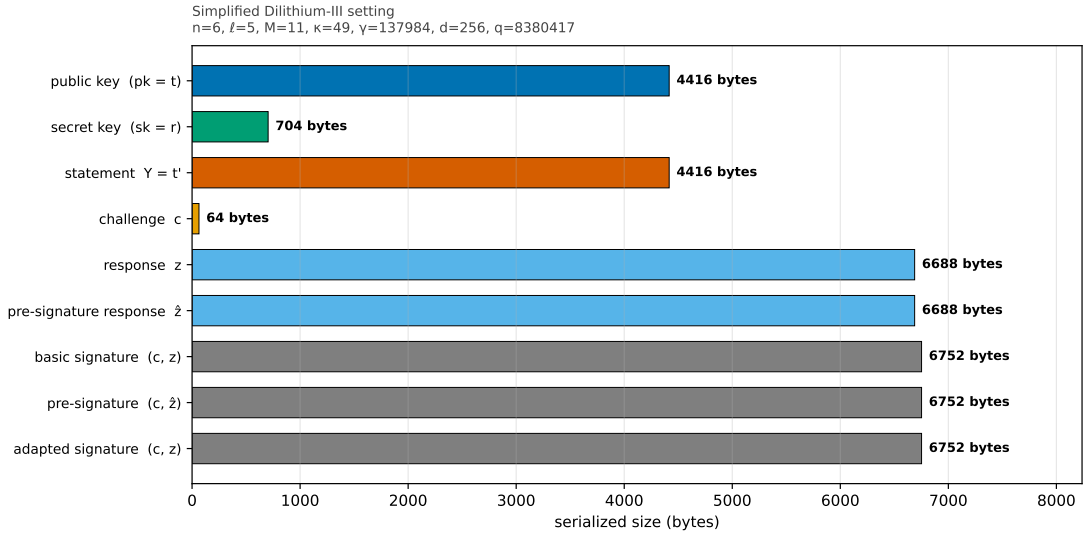

Figure 3.2: Serialized size of every LAS object at the *Simplified Dilithium-III* (target) setting (wire bytes, not on-chain gas). Three facts read off directly: the signature, pre-signature, and adapted signature are byte-identical (adaptation changes the *value* of the response, not its size); the response z dominates every signature; and the only extra *public* object LAS adds over a basic signature is the statement Y, the size of a public key. Sizes at every parameter set: table A.3 (appendix).

Chapter 4

Evaluation and reflection

This chapter evaluates the work against the objectives set out in section 1.3 and reflects on the soundness of the approach, building on the validity discussion in section 3.6.

4.1 Evaluation against the objectives

Each objective from section 1.3 is judged in turn against the evidence in chapter 3.

1. *Literature and scheme selection — met.* The adaptor signature was selected as a representative exotic scheme with high blockchain value, and LAS [5] as the design with the clearest path from a standardised lattice signature, following the survey's framing of the gap [3] and the supervisor's steer toward the simplest viable construction.

2. *Additive implementation — met.* The lattice core is reused byte-for-byte (table 2.1); zero upstream functions were modified, which the two-branch diff makes auditable. The adaptor layer, wire encoding, and application are new code.

3. *Validation — met.* Functional, serialization, tamper, and known-answer tests all pass (section 3.1), establishing the adaptor contract, wire-encoding integrity, and reproducibility. Beyond the single-codebase tests, an independent Rust implementation reproduces the wire format and the pinned known-answer digest byte-for-byte (section 3.3), the strongest correctness evidence available short of a formal proof.

4. *Fair benchmarking — met.* The primary comparison holds algorithm, parameters, and primitives fixed between LAS and its own simplified base, so the primary comparison isolates the adaptor overhead — the base-signature path against the LAS adaptor path, every operation timed independently. The adaptor overhead is reported per operation as the headline (table A.2), with the absolute timings and a parameter-sensitivity check across four settings shown as supporting context (figs. 3.1 and A.1, exact mean $\pm$ standard-deviation figures in table A.1); computation and communication are separated and broken down by component. The optimised CRYSTALS-Dilithium reference is included only as context (not algorithm-matched), and a classical ECDSA adaptor as a functionality-matched "price of post-quantum" baseline, each with the standard five metrics. The measurements themselves are defended two ways: every run gates on a directly counted rejection rate matching its closed-form prediction, and the Rust port re-measures the same operations under a different compiler and an independent statistical harness, reproducing every relative conclusion (table 3.2).

5. *Atomic-swap demonstration — met at the simulated-ledger level.* An end-to-end two-party, two-chain swap settles both legs and asserts unspendability before adaptation (section 3.5). A full deployment on a live testnet was not in scope; an on-chain gas floor and a multi-hop variant exist as bonus-tier evidence.

4.2 Reflection on the approach

Five reflections stand out. First, reuse was the right strategy: building additively on a standardised reference made the implementation tractable in the available time and produced an unusually strong trust story, since the security-critical arithmetic is exactly the audited upstream code. Second, the main engineering subtlety was the PreSign bound budget (eq. (2.1)); getting it wrong does not crash the program but makes every adapted signature fail verification, so it had to be reasoned through rather than discovered by testing alone. Third, measuring the rejection rate directly, by counting attempts, corrected an earlier figure that had been estimated from a biased timing ratio — a concrete lesson in preferring direct measurement to a proxy; that lesson is now institutionalised as a gate every benchmark run must pass. Fourth, the second implementation repaid

its cost: because the reuse strategy transfers across languages, the Rust port was comparatively cheap to build, yet it upgraded the correctness argument from "one tested codebase" to "two independent codebases that agree byte-for-byte" and exposed the timing conclusions to an independent harness. Finally, confronting the fairness problem honestly, by stating every scheme's parameters and comparing LAS to a parameter-matched base, strengthens the conclusions rather than weakening them: the size gap to optimised Dilithium is shown to be a packing gap, not a dimension gap.

4.3 Limitations

The limitations follow from the validity discussion in section 3.6 and are restated compactly here. There is no formal security analysis of the chosen parameter set, by design. The modulus is $q \approx 2^{23}$ rather than the paper's 2^{24}. The witness is extracted exactly, sidestepping the paper's relaxed, noisy-witness setting and the witness growth it implies for long channels. And the simplified, hint-free scheme produces larger keys and signatures than optimised Dilithium. Each of these is a deliberate scope choice that trades cryptographic optimisation for a transparent, testable artefact, and each maps directly to an item of future work in section 5.2.

Chapter 5

Conclusion and future work

5.1 Conclusions

This dissertation set out to implement and evaluate a post-quantum exotic signature scheme — a lattice-based adaptor signature reusing the CRYSTALS-Dilithium primitives — and to demonstrate it in a blockchain atomic swap. That aim was achieved. A complete adaptor signature was built additively on the Dilithium reference, with the lattice core reused unchanged and the adaptor layer, wire encoding, and application added as new, tested code. The scheme passes functional, serialization, tamper, and known-answer tests, is corroborated by an independent Rust implementation that reproduces the wire format and the pinned known-answer digest byte-for-byte, and drives an end-to-end two-party, two-chain atomic swap.

The evaluation answers the project's central question — what does adaptor functionality cost over a plain signature, and what does post-quantum operation cost over a classical one — with measured data. The primary fair comparison, LAS against its own simplified base, shows the adaptor layer is almost free in computation: PreSign, PreVerify, and Adapt each add only single-digit overhead (at most +10.0%) over the basic operation they mirror, at every parameter set — a conclusion both implementations reproduce independently — and this is relatively cheaper than the classical ECDSA adaptor, whose discrete-logarithm-equality proof dominates its adaptor cost. The real cost is communication: the signature is 98.6–99.3% lattice response vector, and an adaptor swap additionally transmits a public-key-sized statement. The contextual comparison against the optimised CRYSTALS-Dilithium reference — not algorithm-matched — shows that the

remaining size gap is one of packing and production optimisation, not of security level.

Measured against the objectives, all five were met: the scheme was selected and justified, implemented by reuse, validated, benchmarked fairly against two baselines with the standard metrics, and demonstrated in an application. The principal qualification is scope: the parameter set's concrete security is not formally analysed, and the application runs on a simulated ledger rather than a live testnet.

5.2 Future work

The results point to several well-motivated directions.

- **Tighter packing.** Apply Dilithium-style decomposition and a hint vector to the response, and regenerate the public key from a seed, to close most of the size gap identified in section 3.4 — the single largest practical improvement available.
- **Parameter migration.** Move to the paper's $q \approx 2^{24}$ with a new NTT table, and map the parameters to NIST security levels, reporting before-and-after benchmarks so the security/cost trade-off is explicit.
- **On-chain verification.** Develop the byte-level verifier into a deployed precompile or zero-knowledge circuit, building on the measured gas floor, and benchmark a swap on a live testnet.
- **A new research question.** Extend toward functional adaptor signatures, which would move the project from systems implementation toward a novel construction and a publishable contribution.

Appendix A

Code listings and reproduction

Per the writing guide, code snippets belong in an appendix rather than the body. Only the most representative listings are included here; the full source is in the project repository.

A.1 Building and reproducing the benchmarks

The evaluation is reproducible from the commands below. The upstream Dilithium reference is pinned at the repository's initial commit (`2374d22`); the classical baseline library is the BlockstreamResearch `secp256k1-zkp` `ecdsa_adaptor` module at commit `95b9835`; the Rust port builds on a vendored copy of the `fips204` crate (commit `c948882`). The C toolchain is the system compiler (`cc`, Ubuntu GCC 13.3.0) with `-O3`; the Rust toolchain is stable `rustc` (the committed run used 1.96.0) in release profile.

Listing A.1: Fair benchmark and tests (first-stage evaluation). The suite script runs everything below, saves the logs and parsed tables as a timestamped evidence run, and regenerates every figure, table, and inline number of this report from that run.

```
# the one supported evidence pipeline (all tests + all benchmarks +
    report sync)
scripts/run_benchmark_suite.sh

# or individually:
cd ref
```

```
# fair, parameter-matched base-vs-adaptor benchmark at each parameter
    set
make test/bench_levels_paper && ./test/bench_levels_paper
make test/bench_levels2 && ./test/bench_levels2
make test/bench_levels3 && ./test/bench_levels3 # target setting
make test/bench_levels5 && ./test/bench_levels5
# correctness, serialization/tamper, known-answer tests
make test/test_las3 && ./test/test_las3
make test/test_serde3 && ./test/test_serde3
make test/test_kat3 && ./test/test_kat3
# end-to-end atomic swap
make test/test_swap3 && ./test/test_swap3
```

Listing A.2: Classical adaptor baseline (one-time clone).

```
git clone --depth 1 https://github.com/BlockstreamResearch/secp256k1-
    zkp \
    third_party/secp256k1-zkp # tested at commit 95b9835
cd ref && make test/bench_classical && ./test/bench_classical
```

The Rust port is reproduced with the standard Cargo tooling; the test gate runs first because the benchmarks refuse to time unverified code, and the known-answer test asserts the same pinned digest as the C implementation (`641a176c...5a19`).

Listing A.3: Rust port: test gate, statistical benchmark, protocol driver, and size report (one suite script, or the four steps individually).

```
# the one supported Rust evidence pipeline (tests + benchmarks + report
    sync)
scripts/run_rust_bench_suite.sh

# or individually:
cd rust/fips204-las
cargo test --lib --tests # gate: includes the pinned KAT digest
cargo bench --bench las_bench # Criterion.rs statistical harness
cargo run --release --example bench_levels # protocol driver (mirrors
    the C driver)
cargo run --release --example size_report # packed component sizes
```

A.2 Full benchmark data tables

The figures in chapter 3 are the primary presentation of the benchmark results; this section gives the exact numbers behind them, produced by the commands above (listing A.1). Table A.1 lists every per-operation timing with its standard deviation (the data plotted in fig. 3.1), and table A.2 derives the per-operation adaptor overhead at the target setting from it — the exact numbers behind fig. 3.1; table A.3 gives the component byte sizes at every parameter set (the target setting is plotted in fig. 3.2); and table A.4 lists every communication object at the target setting with its paper notation.

Figure A.1 is the parameter-sensitivity check referred to in section 3.2: it repeats the primary base-versus-adaptor comparison at all four parameter sets and confirms that the adaptor premium stays in the single digits as the dimensions are scaled up, so the headline result is not specific to the target setting.

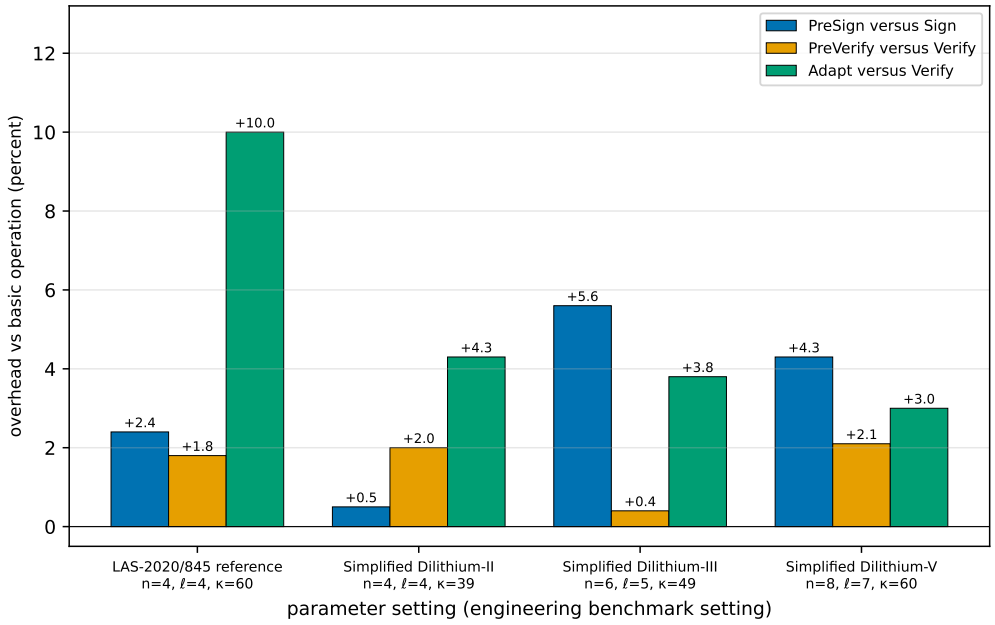

Figure A.1: Parameter sensitivity of the adaptor overhead: each LAS adaptor operation as a percentage over the basic operation it mirrors, at the four parameter sets of table 2.2. PreSign is paired with Sign; PreVerify and Adapt with Verify; Extract has no basic analogue and is omitted. The premium stays in the single digits everywhere, so adding adaptor functionality does not become more expensive as the scheme is scaled up. This is a secondary robustness check; the primary comparison is the per-operation overhead at the target setting, table A.2.

Table A.1: Per-operation timing, reported independently (μs per operation; mean $\pm$ sample standard deviation over 5 repetitions $\times$ 500 sign-class / 1000 verify-class iterations; `bench_levels`, machine of record in section 2.6). KeyGen, Sign, and Verify *are* the basic signature and are reused unchanged by LAS; PreSign, PreVerify, Adapt, and Extract are the four adaptor operations. Public-parameter Setup, a one-time global cost, runs in 23–79 μs and is omitted. Parameter sets and symbols: tables 2.2 and 2.3.

Operation	LAS-2020/845 reference $(4, 4, 60)$	Simplified Dilithium-II $(4, 4, 39)$	Simplified Dilithium-III $(6, 5, 49)$	Simplified Dilithium-V $(8, 7, 60)$
Basic signature (reused unchanged by LAS)				
KeyGen	30.4 ± 0.1	29.9 ± 0.1	45.2 ± 0.2	63.1 ± 0.4
Sign	320 ± 15	351 ± 8	520 ± 21	612 ± 14
Verify	74.6 ± 0.5	73.0 ± 0.9	115 ± 1	153 ± 6
LAS adaptor operations				
PreSign	328 ± 6	353 ± 12	549 ± 31	638 ± 35
PreVerify	75.9 ± 0.8	74.4 ± 0.5	115 ± 2	156 ± 4
Adapt	82.1 ± 7.3	76.1 ± 0.2	119 ± 2	157 ± 1
Extract	26.2 ± 0.1	25.0 ± 0.1	40.9 ± 0.5	55.2 ± 0.4

A.3 Auditing the reuse claim

The "reused vs. modified vs. added" claim (table 2.1) is verified by diffing the pristine baseline branch against the project branch.

Listing A.4: Branch diff proving the lattice core is untouched (no output means identical).

```
git diff --name-status dilithium-baseline main -- ref/
git diff dilithium-baseline main -- ref/poly.c ref/ntt.c ref/sign.c \
    ref/packing.c ref/polyvec.c ref/reduce.c ref/rounding.c ref/params.h
```

A.4 Selected code listing: Adapt and Extract

The adaptor mechanism reduces to two short routines. Adapt adds the witness to the pre-signature's response; Extract recovers the witness by subtraction and confirms it against the statement. Both reuse the reference's polynomial arithmetic verbatim.

Listing A.5: `las_adapt` and `las_ext` from `ref/las.c`.

Table A.2: The per-operation adaptor overhead at the target setting *Simplified Dilithium-III* — the exact numbers behind fig. 3.1. Each LAS adaptor operation is paired with the basic operation it mirrors, and "Overhead" is the extra cost as a percentage of that basic operation. Timings are μs per operation (mean $\pm$ sample standard deviation over 5 repetitions $\times$ 500/1000 iterations; machine of record in section 2.6). KeyGen, Sign, and Verify are reused unchanged by LAS, so their basic and adaptor costs are cycle-identical; Extract has no basic analogue. Absolute timings at every parameter set are in table A.1.

Operation (basic / adaptor)	Basic (μs)	LAS adaptor (μs)	Overhead
KeyGen	45.2 ± 0.2	45.2 ± 0.2 (shared)	—
Sign / PreSign	520 ± 21	549 ± 31	+5.6%
Verify / PreVerify	115 ± 1	115 ± 2	+0.4%
Verify / Adapt	115 ± 1	119 ± 2	+3.8%
Extract	—	40.9 ± 0.5	(LAS only)

```
int las_adapt(las_sig *sig, const las_sig *presig, const uint8_t *m,
    size_t mlen,
              const las_pk *Y, const las_sk *y, const las_pk *pk, const
                  las_pp *pp) {
  unsigned int j;
  if(las_preverify(presig, m, mlen, Y, pk, pp))
    return -1;
  sig->c = presig->c;
  for(j = 0; j < LAS_M; ++j) { /* z = z_hat + y_witness */
    poly_add(&sig->z[j], &presig->z[j], &y->s[j]);
    poly_reduce(&sig->z[j]);
  }
  return 0;
}

int las_ext(las_sk *y, const las_sig *sig, const las_sig *presig,
           const las_pk *Y, const las_pp *pp) {
  poly Ay[LAS_N];
  unsigned int j;
  for(j = 0; j < LAS_M; ++j) { /* s = z - z_hat */
    poly_sub(&y->s[j], &sig->z[j], &presig->z[j]);
    poly_reduce(&y->s[j]);
  }
```

Table A.3: LAS objects by component (packed bytes; `bench_levels`). The LAS-2020/845 reference set shares the Simplified Dilithium-II dimensions. The challenge c is fixed; the response z grows with $n + \ell$ and dominates the signature at every setting.

Component	Simplified Dilithium-II $(4, 4)$	Simplified Dilithium-III $(6, 5)$	Simplified Dilithium-V $(8, 7)$
c (challenge)	64	64	64
z (response)	4608	6688	9120
Signature (c, z)	4672	6752	9184
z as % of signature	98.6%	99.1%	99.3%
Public key pk	2944	4416	5888
Secret key sk	512	704	960
Statement Y (LAS only)	2944	4416	5888
Witness y (LAS only)	512	704	960
Pre-signature (LAS only)	4672	6752	9184

```
  las_Amul(Ay, pp, y->s); /* check A*s == Y */
  for(j = 0; j < LAS_N; ++j)
    if(!poly_equal(&Ay[j], &Y->t[j]))
      return -1;
  return 0;
}
```

Table A.4: Every communication object at the *Simplified Dilithium-III* (target) setting (n=6, ℓ=5, M=11, κ=49, γ=137 984, d=256, q=8 380 417), in packed bytes, with its paper notation. "In the basic signature?" marks which objects LAS shares with the base scheme and which it adds. The adapted signature is itself a valid basic signature. Per-operation timings for this setting are the body's primary table, table A.2.

Object	Notation	Bytes	In the basic signature?
Public key	$pk = t$	4416	yes (shared)
Secret key	$sk = r$	704	yes (shared)
Challenge	c	64	yes
Response	$z \,/\, \hat{z}$	6688	yes
Signature	(c, z)	6752	yes (basic signature)
Statement	$Y = t'$	4416	LAS only (public)
Witness	$y = r'$	704	LAS only (private)
Pre-signature	$(c, \hat{z})$	6752	LAS only
Adapted signature	(c, z)	6752	LAS (verifies as basic sig)

Bibliography

[1] Lukas Aumayr, Oguzhan Ersoy, Andreas Erwig, Sebastian Faust, Kristina Hostáková, Matteo Maffei, Pedro Moreno-Sanchez, and Siavash Riahi. Generalized channels from limited blockchain scripts and adaptor signatures. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 635–664. Springer, 2021.

[2] Blockstream Research. secp256k1-zkp: Ecdsa adaptor signature module. Software library, 2024. `https://github.com/BlockstreamResearch/secp256k1-zkp`, commit 95b9835.

[3] Maxime Buser, Rafael Dowsley, Muhammed Esgin, Clémentine Gritti, Shabnam Kasra Kermanshahi, Veronika Kuchta, Jason Legrow, Joseph Liu, Raphaël Phan, Amin Sakzad, et al. A survey on exotic signatures for post-quantum blockchain: Challenges and research directions. *ACM Computing Surveys*, 55(12):1–32, 2023.

[4] Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, Peter Schwabe, Gregor Seiler, and Damien Stehlé. Crystals-dilithium: A lattice-based digital signature scheme. *IACR transactions on cryptographic hardware and embedded systems*, pages 238–268, 2018.

[5] Muhammed F Esgin, Oğuzhan Ersoy, and Zekeriya Erkin. Post-quantum adaptor signatures and payment channel networks. In *European Symposium on Research in Computer Security*, pages 378–397. Springer, 2020.

[6] Ruslan Kysil, István András Seres, Péter Kutas, and Nándor Kelecsényi. poqeth: Efficient, post-quantum signature verification on ethereum. In *Proceedings of the 20th ACM Asia Conference on Computer and Communications Security*, pages 327–343, 2025.

[7] Vadim Lyubashevsky. Lattice signatures without trapdoors. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 738–755. Springer, 2012.

[8] Giulio Malavolta, Pedro Moreno-Sanchez, Clara Schneidewind, Aniket Kate, and Matteo Maffei. Anonymous multi-hop locks for blockchain scalability and interoperability. In *26th Annual Network and Distributed System Security Symposium, NDSS 2019*, 2019.

[9] National Institute of Standards, Technology (NIST), Thinh Dang, Jacob Lichtinger, Yi-Kai Liu, Carl Miller, Dustin Moody, Rene Peralta, Ray Perlner, and Angela Robinson. Module-lattice-based digital signature standard, 2024-08-13 04:08:00 2024.

[10] Andrew Poelstra. Scriptless scripts. Presentation, Milan Bitcoin meetup / Blockstream, 2017. `https://github.com/apoelstra/scriptless-scripts`.

[11] Peter W Shor. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM review*, 41(2):303–332, 1999.